

CANLI TOMBALA HUB

Mimari ve Teknik Dökümantasyon Raporu

1. Proje Özeti

Canlı Tombala Hub, modern web teknolojileri kullanılarak geliştirilmiş, gerçek zamanlı, ölçeklenebilir ve kriptografik olarak güvenli bir şans oyunları platformudur. Sistem, monolitik bir mimariden olay güdümlü (event-driven) ve bağımsız iş parçacığı tabanlı (threading) bir mimariye evrilmiş; dış kütüphane kilitlemeleri (Eventlet, Celery, Redis) tamamen ortadan kaldırılarak saf Python 3.13 standartlarında kararlı bir sunucu oluşturulmuştur.

2. Sistem Mimarisi ve Altyapı

- Backend:** Python 3.13, Flask, SQLAlchemy (SQLite), Flask-SocketIO, Flask-CORS.
- Frontend:** HTML5, CSS3 (Glassmorphism, Neon Dark Tasarım), Vanilla JS, Socket.IO Client.
- Asenkron İşlem Yönetimi:** Uygulama, oyun döngüsünü ana Flask thread'inden bağımsız, yerel `threading.Thread` yapısı ile arka planda daemon olarak yönetir. Böylece Web sunucusu hiçbir istekte kilitlemez.
- Eşzamanlılık & Uyum:** Eventlet ve SSL uyumsuzlukları giderilerek `async_mode='threading'` parametresiyle WSGI ve WebSocket arasındaki en güvenli köprü oluşturulmuştur.

3. Öne Çıkan Profesyonel Özellikler

Sunucu Tarafı Hile Koruması (Anti-Cheat)

Kullanıcıların tarayıcı üzerinden oyun matrisini (DOM) manipüle etmesini engellemek için tüm işaretleme işlemleri (`/api/mark_number`) ve kazanım talepleri (`/api/claim_win`) kesinlikle sunucu tarafında doğrulanır. İstemciden gelen iddialar red edilir; karar doğrudan veritabanındaki yetkili ve güvenli şema üzerinden verilir.

Provably Fair (Kriptografik Adil Oyun)

Her oyun odası oluşturulduğunda, `secrets` ve `hashlib` kullanılarak benzersiz bir SHA-256 zinciri (`game_hash` ve `game_salt`) üretilir. Çekilen sayıların tamamen şansa dayalı olduğu ve sistem tarafından taraf tutulmadığı matematiksel temellere dayandırılır.

Çoklu Bilet & Oto-İşaretleme (Auto-Daub)

Sistem JSON dizilerini kullanarak tek bir oyuncuya sınırsız sayıda kart tahsis edebilir. Oyuncuya kendi stratejisine göre "Manuel İşaretleme" (heyecan ve odaklanma unsuru) veya "Oto-İşaretleme" seçenekleri sunularak dinamik bir kullanıcı arayüzü sağlanmıştır.

Gerçek Zamanlı Sosyal Lobi (Canlı Chat)

WebSocket odaları (Rooms) aracılığıyla oyuncular sadece aynı oyunu paylaştıkları rakipleriyle uçtan uca gecikmesiz sohbet edebilirler. Oyun içi sosyal etkileşim sürekli canlı tutulur.

4. Veritabanı Şeması (SQLite)

Oyun verilerini kalıcı ve güvenli tutmak için iki ana tablo oluşturulmuştur:

Game Tablosu (Oyun Odaları)

Sütun Adı	Veri Tipi	Açıklama
id	Integer (PK)	Odanın benzersiz referans kimliği
status	String	Oyun durumu ('waiting', 'active', 'finished')
drawn_numbers	JSON	Sunucu tarafından canlı çekilmiş sayıların geçmişi
card_price	Float	Bilet fiyatı (Oyuncunun bakiyesinden tahsil edilecek tutar)

game_hash	String	Adil oyun için kriptografik SHA-256 doğrulama özeti
game_salt	String	Tahmin edilemezliği garantileyen güvenlik tuzu (salt)

Player Tablosu (Oyuncu Profilleri)

Sütun Adı	Veri Tipi	Açıklama
id	Integer (PK)	Oyuncunun sistem içi benzersiz referans kimliği
username	String	Kullanıcı adı (Platform içerisinde unique olmalıdır)
cards	JSON	Kullanıcının satın aldığı 3x5 matris formatındaki biletler dizisi
marked_numbers	JSON	Sunucu onaylı işaretlenmiş sayılar (Hile önleme referansı)
balance	Float	Cüzdan bakiyesi (Otomatik kart tahsilat sistemi için entegre)

5. HTTP API Endpoint Dökümü

- **POST /api/admin/start_game:** Yeni bir oyun odasını kurar. **card_price** parametresini alır, kriptografik anahtarları (**game_hash**) oluşturur ve bilet satışlarına olanak tanımak için 10 saniyelik lobi bekleme (waiting) süresi ile asenkron arka plan işleyişini tetikler.
- **POST /api/buy_ticket:** İstemciden gelen bilet taleplerini işler. Oyuncunun bakiyesini ve oyunun güncel durumunu denetler; işlem başarılıysa güvenli ve yepyeni sayı matrisleri üreterek profiline bağlar.
- **POST /api/mark_number:** Otonom ve manuel kart işaretleme operasyonlarını gerçekleştirir. Sayının gerçekten çekilmiş havuzda (**drawn_numbers**) yer alıp almadığını kontrol ederek işaretleme dizisine ekler.
- **POST /api/claim_win:** İstemciden gelen 'Çinko' veya 'Tombala' kazanım beyanlarını denetler. Analiz sonucunda doğrulanan kazanımların onayı istemci tarafına iletilir.

6. WebSocket (Gerçek Zamanlı) Olayları

Oyunun anlık senkronizasyonu aşağıdaki Socket.IO event'leri ile sağlanır:

- **Event:** **join** (İstemciden Sunucuya) Oyuncuyu belirtilen oyun odasına (Room) dahil eder.

- **Event:** `new_number` (Sunucudan İstemciye) Arka planda çekilen her yeni sayıyı ve güncel sayı geçmişini odaya fırlatır.
- **Event:** `send_message` / `receive_message` İstemciler arası anlık sohbet verisini JSON formatında dağıtır.
- **Event:** `status` (Sunucudan İstemciye) Lobiye katılım durumlarını ve sunucu bilgilendirme mesajlarını iletir.

7. Teknik Kilitlenme & Çözüm Notları

Geliştirme sürecinde tespit edilen dar boğazlar ve alınan önlemler:

- **Python 3.13 ssl.wrap_socket Uyumsuzluğu:** Flask-SocketIO içerisinde eventlet bağımlılığı devre dışı bırakılmış ve sistem lokal iş parçacıklarına (threading) geçirilmiştir.
- **CORS Preflight Bloklaması:** Tarayıcı OPTIONS taleplerinin POST verilerine engel olması Flask-CORS entegrasyonu ile küresel (global) izine dönüştürülmüştür.
- **Veritabanı Tablo Kilitlenmesi (Table Locking):** SQLAlchemy yapılandırmasında Absolute Path (Kesin yol) kullanılmış, güncellemelerde schema crash olmaması için `db.drop_all()` kurgusu geliştirme safhasına eklenmiştir.